Microsoft Teams Security & Compliance — Information Barriers, Retention & DLP

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary 4

1. Solution Overview — Microsoft Teams Security & Compliance: Information Barriers, Retention & DLP 4

What this solution is . . . 4

How it works . . . 5

The reference design (named artifacts) . . . 5

What you can control . . . 5

Benefits . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 6

2. Requirements Guide 7

1. Compliance licensing for Information Barriers and Teams DLP . . . 7

2. Least-privilege compliance roles . . . 7

3. Security & Compliance PowerShell connectivity (Connect-IPPSSession) . . . 7

4. A directory attribute to define IB segments — governed by compliance . . . 8

5. Scoped directory search enabled in Teams (IB dependency) . . . 8

6. No conflicting address book policies (IB dependency) . . . 9

7. Existing Purview posture (retention / DLP) — align, don't collide . . . 9

8. Compliance, records & legal stakeholders and their determinations . . . 9

Global Requirements — Microsoft Teams Solutions 10

1. Microsoft 365 tenant with Microsoft Teams and appropriate licensing . . . 10

2. Administrative access (least-privilege Teams roles) . . . 10

3. Microsoft Teams PowerShell + Graph toolchain . . . 10

4. Identity, groups & lifecycle source of truth . . . 11

5. A pilot scope and change control . . . 11

6. Security, compliance & governance stakeholders . . . 11

3. Implementation Guide 12

Stage 0 — Assess (read-only) . . . 12

Stage 1 — Design (on paper) . . . 12

Stage 2 — Configure Information Barriers . . . 13

Stage 2 — Configure Teams Retention . . . 13

Stage 2 — Configure Teams DLP (in test mode) . . . 14

Stage 3 — Pilot & Validate (per control, with gates + rollback) . . . 14

Stage 4 — Expand . . . 15

Rollback summary (per control) . . . 15

5. Testing & Validation 16
Test 1 — IB segments and policy exist and are Active . . . 16
Test 2 — IB application completed . . . 16
Test 3 — IB blocks communication (functional, in the Teams client) . . . 16
Test 4 — Teams retention policy enabled with correct rule . . . 16
Test 5 — Teams DLP policy present with the audit/enforce mode intended . . . 17
Test 6 — Teams DLP functional check (test then enforce) . . . 17
Test 7 — Automated validation report . . . 17
Test 8 — No collision with the broader Purview program . . . 17
6. Troubleshooting 18
A. Information barriers . . . 18
B. Teams retention . . . 19
C. Teams DLP . . . 20
D. Cross-cutting . . . 20
7. Best Practices 21
Scope and alignment . . . 21
Determinations stay with compliance/legal . . . 21
Information barriers . . . 21
Teams retention . . . 22
Teams DLP . . . 22
Operations, safety, and evidence . . . 22
8. Deliverables Summary 22
Documentation . . . 22
PowerShell scripts (5) — Security & Compliance PowerShell / MicrosoftTeams / Graph . . . 23
Appendix A — PowerShell Scripts 25

Executive Summary

Outcome. Microsoft Teams is brought under the compliance controls that are specific to it — **information barriers** enforced between groups that must stay separated, **retention** applied to Teams chat and channel messages, and **Data Loss Prevention** effective in Teams conversations. These are the Teams-specific controls most Purview rollouts leave open (email and files are covered; Teams chat often is not). The result closes that gap and aligns Teams to the organization's existing Purview program rather than duplicating it.

Approach. The solution provides the technical method — exact Security & Compliance PowerShell and Purview-portal steps, five safe-by-default PowerShell tools, and named, reusable artifacts — for three controls: information-barrier **segments and policies**, Teams **chat/channel retention**, and Teams **DLP**. The determinations that drive them (who must not communicate, how long to retain, what to block) remain with the organization's **compliance, records, and legal** functions; the solution implements those decisions, it does not make them.

Why it is low-risk. Every control follows a pilot-first model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — with explicit pass/fail gates and a documented rollback per control. Information barriers are validated on a limited pilot population before the attribute is expanded; DLP runs through an **audit gate** in test mode before it ever blocks a message; retention is validated on a small scope before broad deletion. Automation is read-only or `-WhatIf` by default, requires an explicit `-Commit` to change anything, and carries no credentials in code. Plan-, version-, and cloud-dependent specifics (E5/E5-Compliance entitlements, channel-type coverage, Government-cloud endpoints) are flagged to verify in-tenant rather than assumed.

Bottom line. Teams brought under compliance — barriers enforced, chat/channel retention applied, Teams DLP effective — the Teams-specific controls closed and aligned to Purview, delivered as a self-guided, tested runbook at a fraction of consulting cost.

1. Solution Overview — Microsoft Teams Security & Compliance: Information Barriers, Retention & DLP

What this solution is

Microsoft Teams carries chats, channel messages, and files that fall under the same compliance obligations as email and documents — but the **Teams-specific** controls are easy to miss when an organization stands up a broad Microsoft Purview program. This solution applies the compliance controls that are **specific to Teams**:

1. **Information barriers (IB)** — policies that stop two defined groups of people from communicating in Teams (chat, calls, meeting invites, and being added to the same team). Used where regulation or policy requires an "ethical wall" (for example a group that must not talk to another group).
1. **Teams retention** — retention policies scoped to **Teams chat messages** and **Teams channel messages (and, where used,** private-channel **and** shared-channel** messages), so Teams conversations are kept for a required period and then deleted, or blocked from deletion.

1. **Teams DLP** — Data Loss Prevention policies scoped to the **Teams** location, which detect

sensitive content in Teams chats and channel messages and audit, warn on, or block it.

This solution deliberately covers **only the Teams-specific slice**. The broad Purview program — the sensitivity-label taxonomy, org-wide DLP across Exchange/SharePoint/OneDrive/endpoint, retention labels and records management — is referenced, not rebuilt. If your organization already runs a Purview program, this solution extends it into Teams; if it does not, the Requirements Guide points to that broader work.

How it works

1. **Information barriers** are built from **segments** (groups of users defined by a directory

attribute — Department, `MemberOf`, and so on) and **IB policies** that either *block* or *allow* communication between segments. You define the segments, define the policies, then run a tenant **application** step that evaluates every user against the segments and enforces the barriers. Barriers show up in Teams as blocked chats/searches and prevention from joining the same team.

1. **Teams retention** uses a retention policy whose **locations are the Teams locations only**

(Teams chat, Teams channel messages). A retention **rule** on that policy sets the duration and the end-of-life action (retain, retain-then-delete, or delete-only). Teams retention writes to the hidden `SubstrateHolds` / mailbox folders that back Teams messages.

1. **Teams DLP** uses a DLP policy scoped to the **Teams** location with a rule that matches

sensitive-information types or sensitivity labels. As with all DLP, you deploy it first in a **test / audit** mode, review the matches (the **DLP audit gate**), tune out false positives, then switch to enforce (notify or block).

1. Every artifact is created with a **consistent name** and validated on a **pilot scope** before it

is expanded, with a documented rollback at each gate.

The reference design (named artifacts)

The implementation creates and reuses these exact names (yours will differ by content, but the naming convention `<Purpose> – <Scope> – <Stage>` is the point):

- Segments: `IB Segment – GroupA – Prod`, `IB Segment – GroupB – Prod`
- IB policy: `IB Policy – GroupA-blocks-GroupB – Pilot`
- Retention: policy `Retention Policy – Teams Chat+Channel – Pilot`, rule `Retention Rule – Teams 7yr – Pilot`
- Teams DLP: policy `DLP Policy – Teams Sensitive – Pilot`, rule `DLP Rule – Teams PII – Pilot`

What you can control

- **Information barriers:** which segments exist (by attribute), which segments are blocked from or

allowed to communicate, and when the barriers are applied tenant-wide.

- **Teams retention:** the duration, the trigger (when content was created / last modified), and the

end-of-life action for Teams chat and channel messages — including private and shared channels.

• **Teams DLP:** which sensitive-information types or labels a Teams policy matches, whether it runs in test or enforce mode, and the action (audit, notify with a policy tip, or block the message).

Benefits

• **Ethical walls that actually hold** — IB blocks Teams chat, calls, and team membership between groups that must stay separated, enforced by the platform rather than by policy on paper.

• **Defensible Teams retention** — Teams conversations are kept and deleted on a defined schedule, closing the gap most Purview rollouts leave open (email and files retained, Teams chat not).

• **Sensitive content controlled in chat** — Teams DLP catches PII and other sensitive content in chats/channels, tuned in audit before it ever blocks a real conversation.

• **Aligned, not duplicated** — the Teams controls plug into the org's existing Purview program using the same sensitive-information types, labels, and audit surface.

Limitations (be honest with buyers)

• **Information barriers, Teams DLP, and org-wide retention are plan-gated.** IB and Teams DLP typically require the higher Microsoft 365 / Purview compliance plans (E5 / E5 Compliance or the standalone add-on); adaptive-scope retention likewise. The Requirements Guide names the licensing — **verify the exact entitlement in your tenant.**

• **IB application is tenant-wide, not per-pilot.** Once you start IB policy application it evaluates every user. You "pilot" IB by defining the segment attribute so that **only pilot users carry it**, validating, then expanding the attribute's coverage — not by scoping the application run.

• **Teams retention deletion is irreversible** once the period elapses; validate the schedule on a pilot scope first.

• **Records schedules, legal-hold determinations, and the IB policy definitions themselves are compliance/legal decisions, not technical ones. **This solution provides the technical method and the evidence;** the determination of who-must-not-talk-to-whom, and how long to keep what, is deferred to your compliance, records, and legal functions.**

• Teams retention and IB interact with other holds (eDiscovery holds, Litigation Hold); content under a hold is preserved regardless of a delete action. Coordinate with the eDiscovery/legal owners.

The standard deployment model

> ***Assess*** → ***Design*** → ***Configure*** → ***Pilot*** → ***Validate*** → ***Expand***, *pilot-first, with explicit pass/fail gates and a documented rollback. Information barriers, retention deletion, and DLP enforcement are* ***high-impact*** *— each is deployed to a named pilot scope, validated against a gate, and only then expanded. Automation is delivered as* ***PowerShell*** *(Security & Compliance PowerShell / the MicrosoftTeams module / Microsoft Graph), read-only or* `-WhatIf` *by default, with no credentials in*

code.

2. Requirements Guide

These are the **solution-specific** prerequisites for the Teams security & compliance controls. They build **on top of** the shared prerequisites in the category file `global-requirements-teams.md` (tenant + Teams licensing, least-privilege Teams roles, the MicrosoftTeams/Graph toolchain, an identity/group source of truth, a pilot scope + change control, and security/compliance stakeholder engagement) — read that first; the items below add what is specific to information barriers, Teams retention, and Teams DLP. Each item lists **Description / Why / Verify / Implement if missing.**

1. Compliance licensing for Information Barriers and Teams DLP

• **Description:** The Microsoft 365 / Purview plan that entitles **Information Barriers** and **Teams DLP**. **Per Microsoft's public licensing guidance these capabilities are in the higher compliance SKUs (for example Microsoft 365 E5, the** E5 Compliance **add-on, or** E5 Information Protection & Governance **/ Insider Risk Management for IB) —** exact entitlement is plan-dependent; verify in your tenant.**

• **Why required:** Without the entitlement the `New-InformationBarrierPolicy` / `New-DlpCompliancePolicy` operations and the Teams-location DLP will not be available or will not enforce.

• **Verify:** In the Microsoft 365 admin center review the subscriptions and add-ons; in the Purview portal confirm the **Information barriers** and **Data loss prevention** solutions are visible.

• **Implement if missing:** Assign/purchase the required compliance plan or add-on before starting.

2. Least-privilege compliance roles

• **Description:** The Purview role groups the work needs — **Information Barriers** processors for IB (`Get/New/Set-*InformationBarrier*`, segments, and running application), **Compliance Administrator** or **DLP Compliance Management** for DLP, and **Retention Management** for retention. These are separate from the Teams Administrator role in the category GR.

• **Why required:** Segment/IB, DLP, and retention cmdlets run against **Security & Compliance PowerShell** and are gated by these Purview role groups, not by Teams roles.

• **Verify:** In the Purview portal **Roles & scopes** → **Permissions**, confirm your account is in the Information Barriers, Compliance/DLP, and Retention role groups (or a role group that contains them).

• **Implement if missing:** Have a Compliance/Global Admin add your account to the required role groups under change control.

3. Security & Compliance PowerShell connectivity (Connect-IPPSSession)

• **Description:** The **ExchangeOnlineManagement** module, used to reach **Security & Compliance

PowerShell **via** `Connect-IPPSSession` **(this is a *different* endpoint from** `Connect-ExchangeOnline` **and from** `Connect-MicrosoftTeams`**). Government clouds use different connection URIs (for example** `*.office365.us`**) —** flag and verify your cloud's endpoint.**

• **Why required:** All IB, retention, and DLP cmdlets and the four compliance-facing scripts in this

solution run through the Security & Compliance PowerShell endpoint.

• **Verify:** `Connect-IPPSSession` succeeds and `Get-OrganizationSegment` / `Get-DlpCompliancePolicy`

return without an access error.

• **Implement if missing:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser`; for

Government cloud pass the appropriate `-ConnectionUri`/`-AzureADAuthorizationEndpointUri` per Microsoft's cloud-specific docs (verify in your tenant).

4. A directory attribute to define IB segments — governed by compliance

• **Description:** A clean, authoritative user attribute (for example **Department**, **job title**,

or membership of a specific Entra group) that reliably identifies each population the barrier must separate. **Which populations must be separated, and by which attribute, is a compliance/legal determination — not a technical one.**

• **Why required:** IB segments are defined by an attribute filter (`-UserGroupFilter`). If the

attribute is inconsistent or missing, users fall outside every segment and the barrier does not apply to them.

• **Verify:** Confirm the chosen attribute is populated and accurate for the affected users (spot-check

in Entra ID / `Get-MgUser`). Confirm compliance has **defined** the segments and the block/allow relationships in writing.

• **Implement if missing:** Have identity remediate the attribute for the in-scope users; obtain the

segment/relationship definitions from compliance before configuring IB.

5. Scoped directory search enabled in Teams (IB dependency)

• **Description:** Information barriers in Teams require **scoped directory search** to be enabled

org-wide so that people search respects the barriers. This is a Teams **org-wide setting**, set in the Teams admin center (or via configuration), and applies tenant-wide.

• **Why required:** Without scoped directory search enabled, IB cannot fully constrain search/discovery

in Teams and Microsoft's IB enablement steps are incomplete.

• **Verify:** In the **Teams admin center** → **Org-wide settings** → **Teams settings**, confirm the search

scope option; or review the current Teams client configuration.

• **Implement if missing:** Enable scoped directory search per Microsoft's IB prerequisite steps

during a change window (it is tenant-wide) **before** starting IB application — coordinate with the Teams admin owner from the category GR.

6. No conflicting address book policies (IB dependency)

• **Description:** Information barriers are **incompatible with legacy Exchange Address Book Policies (ABPs)** used for address-list segregation. The tenant must not have an ABP-based segmentation that conflicts with IB.

• **Why required:** IB and ABP-based segregation cannot both govern the same directory partitioning; Microsoft's IB docs call out the conflict.

• **Verify:** Check for existing ABPs (`Get-AddressBookPolicy` in Exchange Online PowerShell) and confirm none are relied upon for the populations IB will segment.

• **Implement if missing:** Plan the migration off ABP-based segregation with the messaging owner before enabling IB (this is an existing-environment dependency — surface it early).

7. Existing Purview posture (retention / DLP) — align, don't collide

• **Description:** The current Purview state that Teams controls must fit into: the sensitive-information types / sensitivity labels this Teams DLP will reference, and any existing org-wide retention that already includes or excludes the Teams locations.

• **Why required:** Teams DLP should reuse the org's approved sensitive-info types/labels rather than inventing new ones, and Teams retention must not double-apply against a retention policy that already covers Teams.

• **Verify:** Run the assessment script (or `Get-RetentionCompliancePolicy` / `Get-DlpCompliancePolicy`) to list what already covers the Teams locations; confirm the label/SIT set with the Purview owner.

• **Implement if missing:** Where the broader Purview program is absent, reference the separate Purview program work; do **not** rebuild the label taxonomy or org-wide DLP here.

8. Compliance, records & legal stakeholders and their determinations

• **Description:** Written determinations from the org's **compliance, records, and legal** functions: the IB segment/relationship definitions (item 4), the Teams **retention schedule** (how long, what end-of-life action), and the Teams **DLP intent** (which content, audit vs block). This is the Teams-specific instance of category GR item 6.

• **Why required:** Retention durations, records determinations, legal-hold interactions, and IB policy definitions are **legal/compliance decisions**; the technical steps implement them but must not invent them.

• **Verify:** Confirm named approvers and the written schedule/segment/DLP definitions exist before

configuring.

• **Implement if missing:** Identify the owners and capture the determinations before any policy is created.

Global Requirements — Microsoft Teams Solutions

These requirements apply to **every** solution in the **Teams** category (Teams governance, meetings, external access, Teams Phone, apps, Teams Rooms, and Teams security/compliance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Teams solution).*** *The **Teams admin center** (admin.teams.microsoft.com), the **Microsoft 365 admin center**, the **MicrosoftTeams PowerShell module**, and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Where a capability, policy, or portal path is **plan- or version-dependent** (Teams Phone licensing, Calling Plans vs Operator Connect vs Direct Routing, Premium features, Commercial vs Government cloud), it is **cited or flagged "verify in your tenant"** — never invented. **Third-party devices and apps** (certified Teams Rooms hardware, SBCs, ISV apps) are integrated at the pattern level; their internals are deferred to the vendor's own docs. Automation is **PowerShell** (MicrosoftTeams module / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with Microsoft Teams and appropriate licensing

• **Description:** A Microsoft 365 tenant with **Microsoft Teams** enabled and the licensing the capability requires (e.g. Teams Phone + a calling plan/Operator Connect for voice, Teams Premium for premium features, the appropriate E-plan for advanced compliance).

• **Why required:** All policy, voice, app, and compliance configuration depends on the right plan.

• **How to verify:** In the M365 admin center confirm the Teams subscription and relevant add-ons.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Administrative access (least-privilege Teams roles)

• **Description:** Access with the least-privilege role the solution names — typically **Teams Administrator **(or Teams Communications/Device Administrator), with** Global Administrator** only where genuinely required.

• **Why required:** Teams policy, voice, app, and org-wide settings require the corresponding role.

• **How to verify:** Confirm the role in the M365 admin center / Entra roles.

• **How to implement if missing:** Have a Global Admin assign the least-privilege role under change control.

3. Microsoft Teams PowerShell + Graph toolchain

• **Description:** A workstation with the **MicrosoftTeams** PowerShell module (and Graph PowerShell

where used), modern-auth / MFA-capable admin sign-in.

- **Why required:** Configuration, automation, and the provided scripts run through Teams PowerShell / Graph.
- **How to verify:** `Connect-MicrosoftTeams` succeeds; `Get-Module MicrosoftTeams`.
- **How to implement if missing:** `Install-Module MicrosoftTeams`; confirm modern-auth sign-in.

4. Identity, groups & lifecycle source of truth

- **Description:** A defined identity/group source (Entra ID groups, and where used **Microsoft 365 Groups **which back teams) and the** lifecycle** stance (creation, naming, expiration/retention) the org expects.
- **Why required:** Teams are backed by M365 Groups; governance, provisioning, and access key off groups and the lifecycle model.
- **How to verify:** Confirm which groups/OU model is authoritative and the lifecycle expectations.
- **How to implement if missing:** Define the group/lifecycle model before configuring governance.

5. A pilot scope and change control

- **Description:** A **pilot team / policy group / user set** to validate on before org-wide rollout, and a **change-control** process for production policy/voice/compliance changes.
- **Why required:** Every solution follows an **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model; policy and voice changes are high-impact.
- **How to verify:** Confirm a named pilot scope and an approval/change window are available.
- **How to implement if missing:** Create a pilot scope and agree change control before production changes.

6. Security, compliance & governance stakeholders

- **Description:** Engagement with the **security, compliance/records, and collaboration-governance** stakeholders whose policies the configuration must satisfy (external access, information barriers, retention, DLP, app governance).
- **Why required:** Collaboration governance and compliance settings must align to enterprise policy.
- **How to verify:** Confirm the applicable policies and named approvers.
- **How to implement if missing:** Identify the owners and capture requirements before configuring governance.

*The deployment model across every Teams solution is the same: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **PowerShell** (MicrosoftTeams module / Graph) — never with hardcoded credentials or secrets in code.*

3. Implementation Guide

Follow the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — for each of the three Teams controls. Every artifact is **named** with the convention `<Purpose> – <Scope> – <Stage>` and that exact name is reused in testing, troubleshooting, and the scripts. All PowerShell in this guide runs against **Security & Compliance PowerShell** unless noted; connect once per session:

```
Install-Module ExchangeOnlineManagement -Scope CurrentUser   # if missing
Connect-IPPSSession                                            # Government cloud: add the cloud-specific
```

> ***Determinations come from compliance/legal, not from here.*** *The example segment attributes, retention durations, and DLP content types below are* ***placeholders****. Substitute the written definitions your compliance, records, and legal owners provide (Requirements items 4, 7, 8).*

Stage 0 — Assess (read-only)

1. Run the assessment script to capture today's posture:

`powershell -ExecutionPolicy Bypass -File .\TEAMSCOMP-Assessment.ps1`. It reports existing IB segments/policies and application status, retention policies that already touch the Teams locations, Teams-location DLP policies (with mode), and whether scoped directory search is enabled.

1. Confirm the Requirements Guide items — especially compliance licensing (item 1), roles (item 2),

the segment attribute (item 4), scoped directory search (item 5), no conflicting ABPs (item 6), and the written determinations (item 8).

1. **Assess gate (pass/fail):** entitlement present, roles held, `Connect-IPPSSession` works, no

conflicting ABP for the target populations, and written determinations in hand. **Fail** → **stop** and resolve before configuring.

Stage 1 — Design (on paper)

1. From compliance's determinations, write down: the **segments** (name + defining attribute), the

block/allow relationships between them, the Teams **retention** duration and end-of-life action, and the Teams **DLP** intent (which sensitive-information types/labels, audit vs block).

1. Fix the names now and reuse them everywhere:

- `IB Segment – GroupA – Prod`, `IB Segment – GroupB – Prod`
- `IB Policy – GroupA-blocks-GroupB – Pilot`
- `Retention Policy – Teams Chat+Channel – Pilot` / rule `Retention Rule – Teams 7yr – Pilot`
- `DLP Policy – Teams Sensitive – Pilot` / rule `DLP Rule – Teams PII – Pilot`

1. Define the **pilot scope**: for retention and DLP, a small pilot user/group set. For IB, arrange

that **only pilot users carry the segment-defining attribute** initially (IB application is tenant-wide — see Stage 3A).

Stage 2 — Configure Information Barriers

> *Do the segment/policy creation with `-WhatIf` first (the config helper defaults to this). Creating segments and policies does **not** enforce anything — enforcement happens only at the **application** step in Stage 3A.*

1. **Enable the IB prerequisites** (once): confirm **scoped directory search** is enabled in Teams

(Requirements item 5) and that no conflicting ABP exists (item 6).

1. **Create the segments** from the compliance-defined attribute:

```
New-OrganizationSegment -Name "IB Segment – GroupA – Prod" -UserGroupFilter "Department -eq 'GroupA'"
New-OrganizationSegment -Name "IB Segment – GroupB – Prod" -UserGroupFilter "Department -eq 'GroupB'"
```

(`-UserGroupFilter` supports attributes such as `Department`, `MemberOf`, `Company` — use the one compliance specified.)

1. **Create the IB policy** that blocks communication between the two segments:

```
New-InformationBarrierPolicy -Name "IB Policy – GroupA-blocks-GroupB – Pilot" `
  -AssignedSegment "IB Segment – GroupA – Prod" `
  -SegmentsBlocked "IB Segment – GroupB – Prod" -State Inactive
```

Create it **Inactive** first; review, then set it Active:

```
Set-InformationBarrierPolicy -Identity "IB Policy – GroupA-blocks-GroupB – Pilot" -State Active
```

(Use `-SegmentsAllowed` instead of `-SegmentsBlocked` if compliance defined an *allow*-list model. A segment can be governed by one policy; verify the block/allow direction in your definition.)

1. **Config gate:** `Get-InformationBarrierPolicy` shows the policy **Active**; `Get-OrganizationSegment`

shows both segments. **No barriers are enforced yet** — that is Stage 3A.

Stage 2 — Configure Teams Retention

> *Teams locations must live in a retention policy of their **own** — a single policy cannot mix the Teams locations with Exchange/SharePoint locations.*

1. **Create the Teams retention policy** scoped to Teams chat and channel messages:

```
New-RetentionCompliancePolicy -Name "Retention Policy – Teams Chat+Channel – Pilot" `
  -TeamsChatLocation "pilotgroup@contoso.com" -TeamsChannelLocation "Pilot Team" -WhatIf
```

Use `-TeamsChatLocation All` / `-TeamsChannelLocation All` only when expanding tenant-wide. For private/shared channels add `-TeamsPrivateChannelLocation` / `-TeamsSharedChannelLocation` per your determination (**verify parameter availability in your tenant** — channel-type coverage is version-dependent).

1. **Create the retention rule** with the compliance-defined duration and action:

```
New-RetentionComplianceRule -Name "Retention Rule – Teams 7yr – Pilot" `
  -Policy "Retention Policy – Teams Chat+Channel – Pilot" `
  -RetentionDuration 2555 -RetentionComplianceAction KeepAndDelete
```

(`-RetentionComplianceAction` = `Keep` (retain-only), `KeepAndDelete` (retain then delete), or `Delete` (delete-only). Duration is in days or `Unlimited`. **The duration and action are compliance's determination.**)

1. **Config gate:** `Get-RetentionCompliancePolicy -Identity "Retention Policy – Teams Chat+Channel – Pilot"`

shows **Enabled**; the rule shows the expected duration/action. Remove `-WhatIf` (or run the config helper with `-Commit`) to apply.

Stage 2 — Configure Teams DLP (in test mode)

1. **Create the Teams DLP policy in test mode** (audit, no user impact):

```
New-DlpCompliancePolicy -Name "DLP Policy – Teams Sensitive – Pilot" `
  -TeamsLocation "pilotgroup@contoso.com" -Mode TestWithoutNotifications -WhatIf
```

(`-Mode` values: `TestWithoutNotifications` and `TestWithNotifications` for the audit gate, `Enable` to enforce, `Disable` to turn off. Scope `-TeamsLocation All` only when expanding.)

1. **Create the DLP rule** referencing the org's approved sensitive-information types (reuse existing

Purview SITs/labels — Requirements item 7):

```
New-DlpComplianceRule -Name "DLP Rule – Teams PII – Pilot" `
  -Policy "DLP Policy – Teams Sensitive – Pilot" `
  -ContentContainsSensitiveInformation @(@{Name="U.S. Social Security Number (SSN)"}, @{Name="Credit Ca
  -NotifyUser Owner -BlockAccess $false
```

Keep `-BlockAccess $false` in test/audit; you enable blocking only after the audit gate passes.

1. **Config gate:** the policy exists in a **Test** mode; the rule references the approved SITs. No

messages are blocked yet.

Stage 3 — Pilot & Validate (per control, with gates + rollback)

3A. Information barriers — apply and validate

1. **Apply** the IB policies (tenant-wide evaluation — this is the enforcing step):

```
Start-InformationBarrierPoliciesApplication
```

1. **Watch application status** until complete:

```
Get-InformationBarrierPoliciesApplicationStatus
```

1. **Validate on the pilot users** (only pilot users carry the segment attribute at this point):

```
Get-InformationBarrierRecipientStatus -Identity groupa-user@contoso.com
```

In the Teams client, confirm a GroupA pilot user **cannot** start a chat with, search for, or add a GroupB pilot user to a team.

1. **IB pass/fail gate:** application status = `Completed`; pilot recipients show the expected policy;

Teams blocks the disallowed communication. **Fail** → **rollback:** set the policy `Inactive` (`Set-InformationBarrierPolicy -State Inactive`) and re-run `Start-InformationBarrierPoliciesApplication` to lift the barrier, then fix segments/attribute and retry.

3B. Teams retention — validate on the pilot scope

1. With the policy scoped to the pilot group, confirm it is **Enabled** and the rule duration/action

are correct (`Get-RetentionComplianceRule`).

1. Validate behavior over the pilot period: a message older than the schedule follows the end-of-life

action; content under an eDiscovery/Litigation hold is preserved regardless (coordinate with the eDiscovery owner).

1. **Retention pass/fail gate:** policy Enabled, rule correct, pilot content behaves per the schedule.

Fail → **rollback:** disable the policy (`Set-RetentionCompliancePolicy -Enabled $false`) — note that disabling stops future action but does not undo deletions already taken, which is why the pilot scope is small and the schedule is validated first.

3C. Teams DLP — audit gate, then enforce

1. Leave the policy in **test** mode and review matches in the Purview **Activity explorer / DLP alerts**

for the pilot scope (**the DLP audit gate**). Tune the rule to remove false positives.

1. When matches are clean, **switch to enforce**:

```
Set-DlpCompliancePolicy -Identity "DLP Policy – Teams Sensitive – Pilot" -Mode Enable
# to also block, update the rule:
Set-DlpComplianceRule -Identity "DLP Rule – Teams PII – Pilot" -BlockAccess $true -NotifyUser Owner
```

1. **DLP pass/fail gate:** in enforce mode a pilot user sending the matched content sees the policy tip

/ block; false-positive rate is acceptable. **Fail** → **rollback:** return the policy to `-Mode TestWithoutNotifications` (or `Disable`) — impact stops immediately.

Stage 4 — Expand

1. **IB:** extend the segment-defining attribute to the full populations (or broaden the

`-UserGroupFilter`) and re-run `Start-InformationBarrierPoliciesApplication`; re-validate recipient status across a broader sample.

1. **Retention:** widen the policy locations to `All` (or the full adaptive scope) and confirm

Enabled.

1. **DLP:** widen `-TeamsLocation` to `All`, keep it in enforce, and monitor alert volume.

2. Re-run `TEAMSCOMP-Validation.ps1` after each expansion and record the result. Keep the pilot

artifacts' names or promote them to `– Prod` per your change control.

Rollback summary (per control)

• **IB:** set the policy `Inactive` and re-run application → barrier lifts.

• **Retention:** `Set-RetentionCompliancePolicy -Enabled $false` → future action stops (past deletions

are not reversible — validate on a small pilot first).

• **DLP:** `Set-DlpCompliancePolicy -Mode TestWithoutNotifications` or `Disable` → enforcement stops

immediately.

5. Testing & Validation

Run these after configuration and before each expansion. Each test states the procedure and the **expected pass/fail** signal. The reusable check is `TEAMSCOMP-Validation.ps1`; the tests below name the exact artifacts from the Implementation Guide.

Test 1 — IB segments and policy exist and are Active

- **Procedure:** In Security & Compliance PowerShell:

`Get-OrganizationSegment | Select Name, UserGroupFilter` and `Get-InformationBarrierPolicy -Identity "IB Policy – GroupA-blocks-GroupB – Pilot" | Select Name, State`.

- **Pass:** both segments `IB Segment – GroupA – Prod` and `IB Segment – GroupB – Prod` are listed with

the expected filter; the policy `State` is **Active**.

- **Fail:** a segment is missing, the filter is wrong, or `State` is `Inactive` → revisit Stage 2 IB.

Test 2 — IB application completed

- **Procedure:** `Get-InformationBarrierPoliciesApplicationStatus`.
- **Pass:** the latest application status is **Completed** with no failed recipients.
- **Fail:** status `Failed/In progress` past the expected window → inspect the error, confirm the

segment attribute is populated, re-run `Start-InformationBarrierPoliciesApplication`.

Test 3 — IB blocks communication (functional, in the Teams client)

- **Procedure:** As a **GroupA** pilot user, try to (a) start a chat with a **GroupB** pilot user,

(b) search for that user, and (c) add them to a team.

- **Pass:** all three are blocked / the GroupB user is not discoverable;
  `Get-InformationBarrierRecipientStatus

-Identity groupa-user@contoso.com` shows the expected policy.

- **Fail:** communication is allowed → confirm scoped directory search is enabled (Requirements item 5),

the users carry the segment attribute, and application completed (Test 2).

Test 4 — Teams retention policy enabled with correct rule

- **Procedure:**

`Get-RetentionCompliancePolicy -Identity "Retention Policy – Teams Chat+Channel – Pilot" | Select Name, Enabled` and `Get-RetentionComplianceRule -Policy`

`"Retention Policy – Teams Chat+Channel – Pilot" | Select Name, RetentionDuration, RetentionComplianceAction`.

- **Pass:** policy `Enabled = True`; rule shows the compliance-defined duration and action (for example `2555` / `KeepAndDelete`); policy scoped to the **Teams** locations only.
- **Fail:** disabled, wrong duration/action, or the policy also contains non-Teams locations → fix per Stage 2 retention (Teams locations must be in their own policy).

Test 5 — Teams DLP policy present with the audit/enforce mode intended

- **Procedure:** `Get-DlpCompliancePolicy -Identity "DLP Policy – Teams Sensitive – Pilot" | Select Name, Mode` and `Get-DlpComplianceRule -Policy "DLP Policy – Teams Sensitive – Pilot" | Select Name, BlockAccess, ContentContainsSensitiveInformation`.
- **Pass (audit gate):** `Mode` is a **Test** value and the rule references the approved SITs with `BlockAccess $false`.
- **Pass (post-gate, enforce):** `Mode = Enable`; `BlockAccess` matches the decision.
- **Fail:** the policy is enforcing before the audit gate passed, or references SITs not approved by the Purview owner → return to test mode and re-run the audit gate.

Test 6 — Teams DLP functional check (test then enforce)

- **Procedure:** In a pilot Teams chat, send a message containing a **synthetic** matched value (for example a test credit-card number from Microsoft's SIT test data — never real PII).
- **Pass (test mode):** the event appears in Purview **Activity explorer / DLP alerts**; the message is **not** blocked.
- **Pass (enforce mode):** the sender sees the policy tip / the message is blocked per `BlockAccess`.
- **Fail:** no event logged in test mode → confirm the `-TeamsLocation` scope covers the pilot chat and the SIT actually matches your test value.

Test 7 — Automated validation report

- **Procedure:** `powershell -ExecutionPolicy Bypass -File .\TEAMSCOMP-Validation.ps1`.
- **Pass:** every expected named artifact (segments, IB policy Active, retention policy Enabled + rule, DLP policy + rule mode) reports **PASS**; scoped directory search reports enabled.
- **Fail:** any **FAIL** line → the message names the artifact and expected state; remediate and re-run.

Test 8 — No collision with the broader Purview program

• **Procedure:** Confirm no **other** retention policy already covers the same Teams locations for the

pilot scope (`Get-RetentionCompliancePolicy` review), and that the Teams DLP reuses approved SITs/labels rather than new ones.

• **Pass:** Teams retention/DLP are additive to the Purview program, no double-coverage conflict.

• **Fail:** overlapping Teams retention or a duplicated label/SIT → reconcile with the Purview owner

before expanding.

6. Troubleshooting

Common failure modes for the Teams-specific compliance controls and how to resolve them. Artifact names match the Implementation Guide.

A. Information barriers

A1. IB cmdlets not found / access denied

• **Symptom:** `New-OrganizationSegment` / `New-InformationBarrierPolicy` are unrecognized, or return an

access-denied error.

• **Likely cause:** Not connected to **Security & Compliance PowerShell** (used `Connect-ExchangeOnline`

or `Connect-MicrosoftTeams` by mistake), missing the **Information Barriers** role group, or the plan does not entitle IB.

• **Resolve:** Connect with `Connect-IPPSSession`; confirm membership in the Information Barriers role

group (Requirements item 2); verify the compliance entitlement (item 1). Government cloud: use the cloud-specific connection URI (verify in your tenant).

A2. IB application stuck or failed

• **Symptom:** `Get-InformationBarrierPoliciesApplicationStatus` shows `In progress` for a long time or

`Failed`.

• **Likely cause:** A prior application run has not finished (only one runs at a time), a segment

attribute is missing/inconsistent, or a conflicting address book policy exists.

• **Resolve:** Wait for the current run to finish before starting another; confirm the segment

attribute is populated for the affected users (use `TEAMSCOMP-ScopeHelper.ps1`); clear conflicting ABPs (Requirements item 6). Re-run `Start-InformationBarrierPoliciesApplication`.

A3. Barrier not enforced in Teams (users can still chat)

• **Symptom:** A GroupA user can still chat with, find, or add a GroupB user after application.

• **Likely cause:** Policy left `Inactive`, application not completed, **scoped directory search** not enabled, users carry no segment attribute, or replication has not propagated yet.

• **Resolve:** Confirm the policy `State = Active` and application `Completed` (Tests 1–2); enable scoped directory search (Requirements item 5); verify recipient status (`Get-InformationBarrierRecipientStatus`); allow time for propagation across the tenant.

A4. A user is blocked from everyone / unexpectedly

• **Symptom:** A user cannot communicate with anyone, or an unexpected pair is blocked.

• **Likely cause:** The user matches more than one segment, or the block/allow direction is inverted in the policy definition.

• **Resolve:** Check the user's resolved segments with `Get-InformationBarrierRecipientStatus`; a user should map to exactly one segment. Confirm the block vs allow model with compliance and correct the policy (`-SegmentsBlocked` vs `-SegmentsAllowed`), then re-apply. **Rollback:** set the policy `Inactive` and re-apply to lift the barrier while you fix it.

B. Teams retention

B1. "Teams locations cannot be combined" error

• **Symptom:** `New-RetentionCompliancePolicy` errors when Teams locations are added alongside Exchange/SharePoint locations.

• **Likely cause:** Teams locations must live in a retention policy of their **own**.

• **Resolve:** Create a separate policy (`Retention Policy – Teams Chat+Channel – Pilot`) that contains only the Teams locations.

B2. Retention not applying to Teams messages

• **Symptom:** Messages are not being retained/deleted per the schedule.

• **Likely cause:** Policy `Enabled = False`, wrong scope (chat vs channel vs private/shared channel), or content is under an eDiscovery/Litigation hold that supersedes deletion.

• **Resolve:** Confirm `Enabled = True` and the correct Teams location scope (Test 4); remember holds preserve content regardless of a delete action — coordinate with the eDiscovery/legal owner. Note Teams retention can take time to take effect after enabling.

B3. Private/shared channel messages not covered

• **Symptom:** Standard-channel messages are retained but private/shared channel messages are not.

• **Likely cause:** The policy did not include `-TeamsPrivateChannelLocation` / `-TeamsSharedChannelLocation`, or those parameters are not available on your tenant's version.

• **Resolve:** Add the channel-type locations your determination requires; **verify parameter

availability in your tenant** — channel-type coverage is version-dependent.

C. Teams DLP

C1. DLP policy not matching anything in Teams

- **Symptom:** No events appear in Activity explorer / DLP alerts when test content is sent.
- **Likely cause:** The `-TeamsLocation` scope does not include the test chat, the referenced

sensitive-information type does not actually match the test value, or the policy is `Disabled`.

- **Resolve:** Confirm the Teams location scope covers the pilot chat; use a **synthetic** value that

genuinely matches the SIT (Microsoft's SIT test data); confirm the policy is in a Test/Enable mode (Tests 5–6).

C2. DLP blocking legitimate messages (false positives)

- **Symptom:** Users report legitimate Teams messages being blocked.
- **Likely cause:** The policy was moved to enforce/block before the audit gate cleared false positives.
- **Resolve:** Return the policy to `-Mode TestWithoutNotifications` (impact stops immediately), tune

the rule (confidence level, count thresholds, exceptions), re-run the audit gate, then re-enforce.

C3. Cannot reference a sensitive-information type or label

- **Symptom:** `New-DlpComplianceRule` fails on the `-ContentContainsSensitiveInformation` value.
- **Likely cause:** The SIT/label name is misspelled or not present in the tenant.
- **Resolve:** Reuse the org's **approved** SITs/labels from the Purview program (Requirements item 7);

confirm the exact name exists before referencing it.

D. Cross-cutting

D1. Connected to the wrong endpoint

- **Symptom:** Compliance cmdlets missing though Teams cmdlets work (or vice versa).
- **Resolve:** IB/retention/DLP need `Connect-IPPSSession` (Security & Compliance PowerShell);

Teams client settings need `Connect-MicrosoftTeams`; ABP checks need `Connect-ExchangeOnline`. They are separate endpoints.

D2. Government cloud path differences

- **Symptom:** Connection or portal URLs from Commercial docs do not work.
- **Resolve:** Government clouds use different endpoints (for example `*.office365.us`) and portal URLs;

use your cloud's connection URI and **verify in your tenant** rather than assuming the Commercial path.

D3. Change appears not to take effect immediately

- **Symptom:** A newly enabled policy or barrier is not visible right away.
- **Resolve:** Purview policy propagation and IB application are asynchronous and tenant-wide; allow time to propagate, then re-validate with `TEAMSCOMP-Validation.ps1` before concluding it failed.

7. Best Practices

Durable practices for the Teams-specific compliance controls. These complement — they do not replace — the org's broader Purview program.

Scope and alignment

- **Stay in the Teams lane.** Configure only what is Teams-specific — information barriers, Teams chat/channel retention, and Teams-location DLP. Reference the org-wide Purview program for the label taxonomy, cross-service DLP, and records management; do not rebuild them here.
- **Reuse approved sensitive-information types and labels.** Teams DLP should match the same SITs/labels the Purview owner already approved, so alerts and coverage are consistent across services.
- **Keep Teams retention in its own policy.** Teams locations cannot share a retention policy with Exchange/SharePoint; keep them separate and name them so the split is obvious.

Determinations stay with compliance/legal

- **Never invent the policy.** Who-must-not-talk-to-whom (IB), how long to keep what (retention), and what content to block (DLP) are **compliance/records/legal determinations**. Capture them in writing before configuring, and treat this solution as the technical implementation of those decisions.
- **Coordinate with holds.** eDiscovery holds and Litigation Hold preserve content regardless of a retention delete action; confirm interactions with the eDiscovery/legal owners before enabling delete-only or retain-then-delete on Teams.

Information barriers

- **Pilot by attribute coverage, not by application scope.** IB application is tenant-wide; validate by having only pilot users carry the segment-defining attribute first, then expand the attribute.
- **Use one clean, authoritative attribute** for segments and keep it accurate — a user with no segment attribute is governed by no barrier.
- **Create policies Inactive, review, then activate** — and apply during a change window, because application evaluates every user.
- **Enable scoped directory search** and clear any conflicting address book policies **before** the first application run.

Teams retention

• **Validate the schedule on a small pilot before broad delete.** Deletion is irreversible once the period elapses; a small scope contains the blast radius while you confirm behavior.

• **Be explicit about the end-of-life action** (`Keep`, `KeepAndDelete`, `Delete`) and the trigger, and record which the compliance schedule specified.

• **Cover the channel types your determination requires** (standard, private, shared) — channel-type parameter availability is version-dependent, so verify in your tenant.

Teams DLP

• **Always run the audit gate first.** Deploy in a Test mode, review Activity explorer / DLP alerts, tune out false positives, and only then enable enforce/block.

• **Notify owners, not everyone, at first.** Start with owner notification and no block; add blocking once the false-positive rate is acceptable.

• **Test with synthetic data only** — never send real PII to validate a DLP rule.

Operations, safety, and evidence

• **Safe-by-default automation.** Run the provided scripts read-only or with `-WhatIf`; the config helpers require an explicit `-Commit` to write. No credentials in code — use modern-auth `Connect-IPPSSession` / `Connect-MicrosoftTeams`.

• **Least privilege.** Use the specific Purview role groups (Information Barriers, DLP/Compliance, Retention) rather than Global Admin.

• **Keep a rollback ready per control** (IB → Inactive + re-apply; retention → disable; DLP → back to Test/Disable) and know its limits (a retention delete already executed is not reversible).

• **Re-validate after every expansion** with `TEAMSCOMP-Validation.ps1` and keep the dated output as evidence of the control's state for audit.

• **Verify plan/cloud specifics in your tenant.** IB and Teams DLP entitlements, channel-type coverage, and Government-cloud endpoints (`*.office365.us`) are plan-/version-/cloud-dependent — confirm, don't assume.

8. Deliverables Summary

Everything included in this solution, on one page.

Documentation

• **Solution Overview** — the three Teams-specific controls (information barriers, Teams chat/channel

retention, Teams DLP), how each works, the named reference design, benefits, and honest limitations, plus how they align to the broader Purview program.

• **Requirements Guide** — solution-specific prerequisites (compliance licensing, Purview role groups,

`Connect-IPPSSession`, the IB segment attribute, scoped directory search, no conflicting ABPs, existing Purview posture, and the compliance/records/legal determinations) — each with Verify / Implement steps, building on the Teams category global requirements.

• **Implementation Guide** — ordered Assess → Design → Configure → Pilot → Validate → Expand steps for

each control, every artifact named, with per-control pass/fail gates and rollback.

• **Testing & Validation** — eight tests, each with an expected pass/fail signal.

• **Troubleshooting** — common failure modes (IB not enforcing, application stuck, retention not

applying, DLP not matching, licensing/role errors) and their resolutions.

• **Best Practices** — durable practices for scope, determinations, IB, retention, DLP, and safe

operations.

• **Executive Summary** — outcome, approach, and why the rollout is low-risk.

PowerShell scripts (5) — Security & Compliance PowerShell / MicrosoftTeams / Graph

All are **safe by default** (read-only or `-WhatIf`; config helpers require `-Commit` to write) and use modern-auth connect cmdlets with **no credentials in code**.

1. **TEAMSCOMP-Assessment.ps1** — *Discovery / posture (read-only).* Inventories the Teams compliance

posture: existing IB segments and policies with application status, retention policies that touch the Teams locations, Teams-location DLP policies with their mode, and whether scoped directory search is enabled. The "what exists today" read used to plan.

1. **TEAMSCOMP-Configure-InformationBarriers.ps1** — *IB config helper (`-WhatIf` by default).* Creates

the named organization segments and the block/allow IB policy from parameters, defaulting to a dry-run; writes only when `-Commit` is passed. Optionally starts policy application with an explicit switch.

1. **TEAMSCOMP-Configure-RetentionDlp.ps1** — *Retention + DLP config helper (`-WhatIf` by default).*

Creates the Teams-only retention policy + rule (compliance-defined duration/action) and the Teams DLP policy + rule in a **test** mode referencing approved sensitive-information types; dry-run unless `-Commit` is passed.

1. **TEAMSCOMP-Validation.ps1** — *Compliance-control validation report (read-only).* Checks the

expected named artifacts and their states — segments present, IB policy Active and application Completed, retention policy Enabled with the correct rule, DLP policy/rule mode — and reports PASS/FAIL

per control.

1. **TEAMSCOMP-ScopeHelper.ps1** — *Scope / health helper (read-only).* Resolves and previews the IB

segment membership for sample users, reports `Get-InformationBarrierRecipientStatus`, and checks the IB prerequisites (scoped directory search, conflicting address book policies) so scope is verified before application.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

TEAMSCOMP-Assessment.ps1

TEAMSCOMP-Configure-InformationBarriers.ps1

TEAMSCOMP-Configure-RetentionDlp.ps1

TEAMSCOMP-ScopeHelper.ps1

TEAMSCOMP-Validation.ps1